

Questions

1. Différence entre système distribué et système parallèle
2. Définir l'architecture du système distribué
3. Les défis et les avantages du système distribué
4. Différence entre système 1 cœur et système multicœurs et système multiprocesseurs
5. Définir le modèle de fonctionnement et modèle de programmation du système distribué
6. Définir les protocoles et leurs fonctionnalités utilisés dans les deux modes de communication (mode connecté TCP et mode non connecté UDP)
7. Définition et types de représentations utilisés pour la sérialisation ?
8. Définition et principe de fonctionnement de middleware par appel de procédures distantes (RPC)
9. Décrire le mécanisme d'échange reposant sur la technique de délégation appelée le design pattern proxy (interface, stub, skeleton, annuaire, ...)
10. Définition et principe du RMI : Décrire le mécanisme d'échange reposant sur la technique de délégation appelée le design pattern proxy (interface, stub, skeleton, annuaire, ...)

Exercice 1 :

On souhaite stocker sur un serveur de données un ensemble de coordonnées 2D décrivant les positions d'un certain nombre de véhicules dans le plan.

Ce serveur RMI devra inclure les fonctionnalités (interfaces) suivantes :

- accès au nombre de positions,
- test de l'existence d'une position possédant un nom particulier,
- accès en lecture aux positions,
- création d'une nouvelle position et attribution d'un nom,
- destruction d'une position,
- accès en écriture aux positions.

Questions :

1. Définir et implémenter ces interfaces
2. Ecrire l'application serveur qui permet de créer et d'exposer l'objet dans l'annuaire.
3. Ecrire une application cliente ayant pour but de créer une nouvelle position sur un poste serveur (considérez le poste serveur initialement vide car venant d'être lancé).
4. Ecrire une application cliente ayant pour but de détruire une position sur un poste serveur.
5. Ecrire une application cliente ayant pour but de modifier une position sur un poste serveur.

Exercice 2 : Sérialisation

Écrire une application permettant de sérialiser les informations relatives à une classe **Point** permettant de représenter un point sur un axe. Chaque point sera caractérisé par un *nom* (de type **char**) et une *abscisse* (de type **double**). On prévoira :

- un constructeur recevant en arguments le nom et l'abscisse d'un point,
- une méthode affiche imprimant (en fenêtre console) le nom du point et son abscisse,
- une méthode translate effectuant une translation définie par la valeur de son argument.

On doit prévoir la désérialisation

Exercice 3 :

Ecrire une application distribuée RMI permettant de vérifier si un nombre entier lu au clavier à partir de poste client est premier.

Indication :

1. Définir l'interface de l'objet et donner son implémentation
2. Créer l'objet puis l'exposé dans l'annuaire. Lancer le serveur.
3. Créer le client.

Bonne chance